

比特币（4）——脚本

脚本

脚本是什么

脚本是一种编程语言或者指令集，用于控制和自动化软件程序的运行。在计算机领域，脚本可以是一段文本文件，包含一系列计算机指令，用于执行特定的任务或操作。脚本通常用于简化复杂任务的执行和管理，自动化重复性工作，并提高工作效率。常见的脚本语言包括Python、JavaScript、Shell等，用于编写各种类型的脚本程序。

比特币使用的脚本语言是基于栈的简单脚本语言，在比特币交易过程中，脚本语言用于定义和验证交易的条件。比特币的脚本语言不支持循环和递归，它主要用于验证交易的输入和输出，并决定是否允许该交易被打包进区块中。在比特币网络中，使用脚本语言可以创建多种类型的交易，如标准支付、多重签名支付、时间锁定交易等，增加了交易的灵活性和安全性。

交易实例

Transaction

View information about a bitcoin transaction

921af728159e3019c18bbe0de9c70aa563ad27f3f562294d993a208d4fcdd24

1MaBFqBEfcQyXPv3fm5WAW9aQuJuKHaA3A (0.76469684 BTC - Output)

➔

19z8LJkNXLrTv2QK5jgTncJCGUEEpQvSr - (Unspent) 0.22684 BTC

1LvGTpdyeVLcLcDK2m9f7Pbh7zwhs7NYhX - (Spent) 0.53756644 BTC

23 Confirmations

0.76440644 BTC

Summary		Inputs and Outputs	
Size	226 (bytes)	Total Input	0.76469684 BTC
Weight	904	Total Output	0.76440644 BTC
Received Time	2018-07-06 03:08:26	Fees	0.0002904 BTC
Included in Blocks	530657 (2018-07-06 03:12:07 + 4 minutes)	Fee per byte	128.496 sat/B
Confirmations	23 Confirmations	Fee per weight unit	32.124 sat/WU
Visualize	View Tree Chart	Estimated BTC Transacted	0.22684 BTC
		Scripts	Hide scripts & coinbase

Input Scripts

ScriptSig: PUSHDATA(72)
[3045022100928496fb0d2a25e4e7c99b9c60d4d0d12fcf8974a0fa9cb30119b0d385872a30220253d3d0c507e5e44e123bc28b795ab4a38bf3b205455403e77aa72d58d9e11
PUSHDATA(33)[022ef8d3a6dd8a7039e513acc8ecf9b094ed7e85439824a1d11920f85927cd0018]

Output Scripts

DUP HASH160 PUSHDATA(20)[628ed6567c0b9056067309f07bba2992ecad743] EQUALVERIFY CHECKSIG

DUP HASH160 PUSHDATA(20)[da7d57dfd02c6f5a9c649e891b5ac199ad012cd2] EQUALVERIFY CHECKSIG

- 1. 里面有一个输入（Output）指的是这个输入中使用的币来源于之前那个交易的输出；
两个输出（Input），我们可以清晰的看出，输入的币被分成了 花出去（Spent）和剩下来（Unspent）两个部分
- 2. 比特币脚本使用的语言，唯一能访问的内存空间只有堆栈，因此这种语言也被称为“基于栈的语言”（stack based language）
- 3. confirmation（确认），指的就是交易所在区块之后的区块个数。

交易结构

交易的具体内容

```
"result": {
  "txid": "921a...dd24",
  "hash": "921a...dd24",
  "version": 1,
  "size": 226,
  "locktime": 0,
  "vin": [...],
  "vout": [...],
  "blockhash": "0000000000000000000002c510d...5c0b",
  "confirmations": 23,
  "time": 1530846727,
  "blocktime": 1530846727
}
```

1. **locktime** 用来设定交易的生效时间。后面是数值表示“等待? 个区块生效”，这里的 0 就意味着立即生效。
2. **vin vout** 指的是输入和输出部分，之后会仔细讲
3. **blockhash** 指的是交易所在区块的哈希值
4. **confirmation** 指的是这个交易所在区块之后的区块个数
5. **time**和**blocktime**指的是从很早之前某个固定的时间到现在的秒数

交易的输入结构

```
"vin": [{
  "txid": "c0cb...c57b",
  "vout": 0,
  "scriptSig": {
    "asm": "3045...0018",
    "hex": "4830...0018"
  },
},
],
```

1. 一个交易可以有多个输入，样例展示的交易只有一个输入
2. **txid** 指的是币来源交易的哈希值
3. **vout** 指的是币的来源交易 是上面txid对应tx的的第几个输出
4. **scriptSig** 输入脚本，最简单的输入脚本只用给出Signature（签名）就可以，来证明你有权花这个钱。之后的scriptSig就写作input script

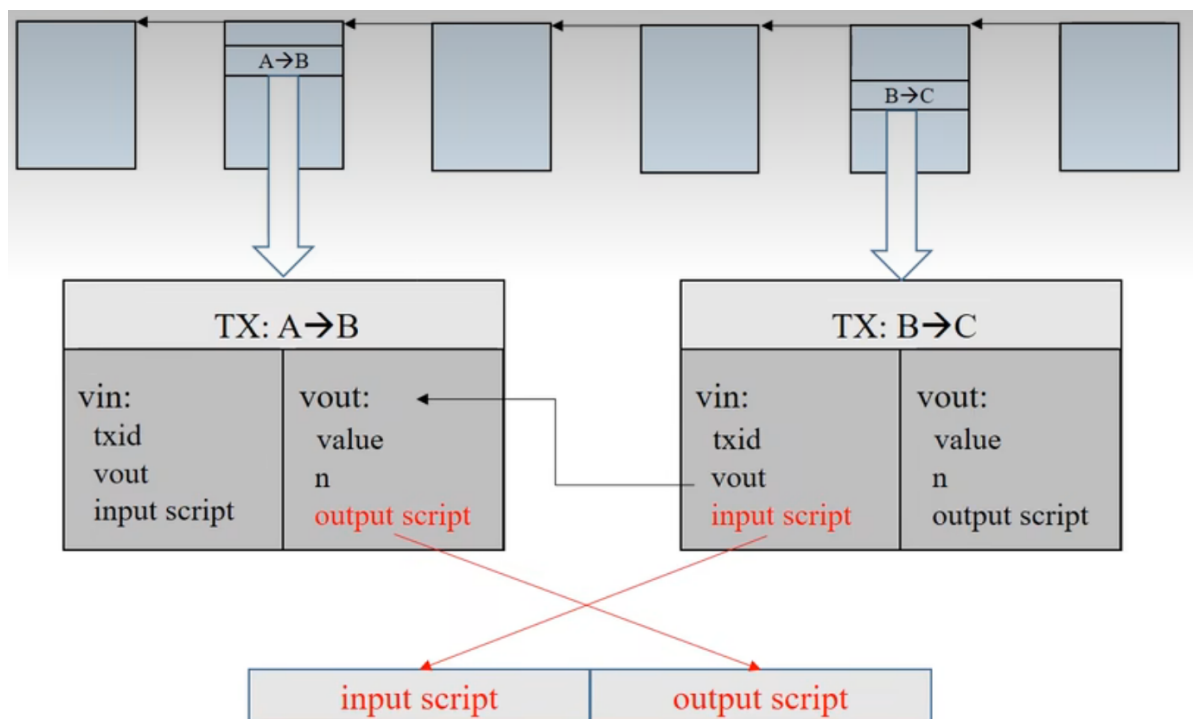
如果一个交易有多个输入的话，就需要多个签名来分别说明币各个来源的合法性。

交易的输出结构

```
"vout": [{
  "value": 0.22684000,
  "n": 0,
  "scriptPubKey": {
    "asm": "DUP HASH160 628e...d743 EQUALVERIFY CHECKSIG",
    "hex": "76a9...88ac",
    "reqSigs": 1,
    "type": "pubkeyhash",
    "addresses": [ "19z8LJkNXLrTv2QK5jgTncJCGUEEfpQvSr" ]
  }
}, {
  "value": 0.53756644,
  "n": 1,
  "scriptPubKey": {
    "asm": "DUP HASH160 da7d...2cd2 EQUALVERIFY CHECKSIG",
    "hex": "76a9...88ac",
    "reqSigs": 1,
    "type": "pubkeyhash",
    "addresses": [ "1LvGTpdyeVLcLCDK2m9f7Pbh7zwhs7NYhX" ]
  }
}],
```

1. 这是上述交易的输出，该数据结构包含两个输出新消息
2. **value** 指的是该输出的金额
3. **n** 指的是这是这个交易里面的第几个输出
4. **scriptPubKey** 指的是输出脚本，输出脚本最简单的形式就是给出一个public key（公钥）
 1. **asm** 显示的是输出脚本里面的内容
 2. **hex** 指的是输出脚本的编码形式
 3. **reqSigs** 指的是这个输出需要多少个签名才能兑现
 4. **type** 指的是输出的类型，这两个输出都是公钥类型
 5. **address** 指的是输出的地址

交易的执行



1. 基于安全因素的考虑，首先执行输入脚本，执行通过后再执行输出脚本，如果执行结果为“true”，则交易是合法的
2. 当有多个输入和输出时，要先进行脚本的匹配，并且分别执行，全部为真值的话，则该交易合法。

交易类型

下方的脚本为了方便都没有使用op_前缀

P2PK(Pay to Public Key)

脚本内容

input script:

PUSHDATA(Sig) 付款人的签名

output script:

PUSHDATA(PubKey) 收款人的公钥

CHECKSIG 验证签名是不是付款人的

脚本执行

实际上是分别执行，这里为了方便写成一条条语句进行执行

PUSHDATA(Sig) 将付款者的签名压入栈

PUSHDATA(PubKey) 将收款人的公钥压入栈

CHECKSIG 将栈顶的两个元素弹出来，用转账人的公钥检验一下签名是不是转账人的。如果是就返回true；如果不是就返回0，这个交易是非法的

实例

交易 [ea44e97271691990157559d0bdd9959e02790c34db6c006d779e82fa5aee708e](#) 的第一个输入:

Input Scripts

```
ScriptSig: PUSHDATA(71)
[30440220576497b7e6f9b553c0aba0d8929432550e092db9c130aae37b84b545e7f4a36c022066cb982ed80608372c139d7bb9af335423d5280350fe3e06bd510e695480914f01]
```

交易 [f4184fc596403b9d638783cf57adfc4c75c605f6356fbc91338530e9831e9e16](#) 的第一个输出:

Output Scripts

```
PUSHDATA(65)[04ae1a62fe09c5f51b13905f07f06b99a2f7159b2225f374cd378d71302fa28414e7aab37397f554a7df5f142c21c1b7303b8a0626f1bade5c72a704f7e6cd84c]
CHECKSIG
```

P2PKH(Pay to Public Key Hash)

这种交易形式是最常用的

脚本内容

input script:

PUSHDATA(Sig) 付款人的签名

PUSHDATA(PubKey) 付款人的公钥

output script:

DUP

HASH160

PUSHDATA(PubKeyHash) 收款人公钥的哈希

EQUALVERIFY

CHECKSIG

脚本执行

PUSHDATA(Sig) 将付款人的数字签名压入栈中, 表示交易的授权

PUSHDATA(PubKey) 将付款人的公钥压入栈中, 用于验证数字签名的有效性

DUP 把栈顶的元素复制一遍再压入栈 (付款人的公钥)

HASH160 将栈顶元素取出, 计算哈希再压入栈 (付款人公钥的哈希)

PUSHDATA(PubKeyHash) (收款人公钥的哈希)

EQUALVERIFY 弹出栈顶的两个元素, 比较他们是否相等, 如果相等则继续执行, 否则中止交易。

这一步目的是确保**付款人正在使用指定的收款人的公钥**进行交易

CHECKSIG 验证签名是不是付款人的

对于EQUALVERIFY 验证机制的解释

在比特币交易中，付款人的公钥哈希值与收款人的公钥哈希值会相等的前提是：

付款人在交易中使用了收款人指定的公钥进行数字签名，而不是使用其他公钥进行签名。这个神奇的性质是比特币交易机制决定的

这是因为交易中会指定收款人的公钥哈希值，而付款人需要使用与之对应的公钥进行数字签名并验证，以确保交易的有效性和安全性。如果付款人使用了其他公钥进行签名，那么计算出的公钥哈希值与指定收款人的公钥哈希值就会不相等，导致交易无效。因此，为了确保交易的正确性，付款人需要使用指定的收款人的公钥进行数字签名，使得这两者的哈希值相等。

实例

交易 [921af728159e3019c18bbe0de9c70aa563ad27f3f562294d993a208d4fcfdd24](#) 的第一个输入：

Input Scripts

```
ScriptSig: PUSHDATA(72)
[3045022100928496fb0d2a25e4e7c99b9c60d4d0d12fcf8974a0fa9cb30119b0d385872a30220253d3d0c507e5e44e123bc28b795ab4a38bf3b205455403e77aa72d58d9e17
PUSHDATA(33)[022ef8d3a6dd8a7039e513acc8ecf9b094ed7e85439824a1d11920f85927cd0018]
```

交易 [c0cb92ca8e41070233bf965d808b0fc4bac144dab05690b17823fac3e184c57b](#) 的第一个输出：

Output Scripts

```
DUP HASH160 PUSHDATA(20)[e1a8cdae6411b17ee1d4cecf47bafe37e14d14] EQUALVERIFY CHECKSIG
```

P2SH(Pay to Script Hash)

1. 这种交易形式给出的不是收款人公钥的哈希，而是收款人公钥的一个脚本的哈希。这个脚本叫做 赎回脚本（redeemScript）。
2. 输入脚本需要给出赎回脚本的内容以及能让赎回脚本正常运行的签名

input script要给出一些签名(数目不定)及一段序列化的redeemScript。验证分如下两步：

1. 验证序列化的redeemScript是否与outputscript中的哈希值匹配？
2. 反序列化并执行redeemScript，验证inputscript中给出的签名是否正确？

redeemScript的形式

1. P2PK形式
2. P2PKH形式
3. 多重签名形式

脚本内容

input script:

...

PUSHDATA(Sig) 付款人的签名

...

PUSHDATA(serialized redeemScript) 赎回脚本

output script:

HASH160

PUSHDATA(redeemScriptHash) 收款人公钥的哈希

EQUAL

脚本执行

使用P2SH实现P2PKH的内容

redeemScript:

PUSHDATA(PubKey) 给出公钥

CHECKSIG 检查签名

input script:

PUSHDATA(Sig) 给出签名

PUSHDATA(serialized redeemScript) 给出序列化的赎回脚本

output script: 验证输入脚本里给出的赎回脚本是否正确

HASH160

PUSHDATA(redeemScriptHash)

EQUAL

执行

第一阶段

PUSHDATA(Sig) 将转账者的签名压入栈

PUSHDATA(seriRS) 将赎回脚本压入栈

HASH160 将赎回脚本取出，并且计算它的哈希值，再压入栈

PUSHDATA(RSH) 将输出脚本给出的哈希值压入栈

EQUAL 判断这两者是否相等，如果相等，那么两者同时消失，第一阶段验证结束。

现在，栈里只剩下了Sig

第二阶段

首先，节点需要将输出脚本提供的程序化赎回脚本进行反序列化

PUSHDATA(PubKey) 将付款者的公钥入栈（现在处于第一阶段的Sig的上面）

CHECKSIG 使用付款者的PubKey，验证签名是不是付款者的

为什么要用P2SH

常见的应用场景就是对多重签名的支持（即一个输出需要多个签名来把币取出来）

比如一个基金有5个所有者，需要三个人的签名才能实施转账。

1. P2SH为防止在这一过程中私钥的泄露造成经济损失提供了保障，即一个人是私钥泄露，坏人无法拿这个私钥进行转账
2. 也为私钥的丢失提供了弥补的机会，哪怕两个人的撕咬丢失，账户上的币依旧可以通过其他三个人的签名进行转移

多重签名实例（不是P2SH）

最早的多重签名，目前已经不推荐使用

input script:

X(任意无意义的元素) 由于check multic数据出栈时存在的一个bug，我们多压入一个元素来解决问题

PUSHDATA(Sig 1)

PUSHDATA(Sig 2)

...

PUSHDATA(Sig M)

outputScript:

M (指的是在所有签名中，只要有M个签名被验证就可以实现合法转账)

PUSHDATA(pubkey 1) 给出的M个签名的相对顺序需要和N个公钥的相对顺序是一致的才行

PUSHDATA(pubkey 2)

...

PUSHDATA(pubkey N)

N (总私钥个数)

CHECKMULTISIG

脚本执行

相对顺序!

1

FALSE (无意义元素)

PUSHDATA(Sig 1)

PUSHDATA(Sig 2)

2 (阈值M)

PUSHDATA(pubkey 1)

PUSHDATA(pubkey 2)

PUSHDATA(pubkey 3)

3 (总私钥个数N)

CHECKMULTISIG 执看看是否有M个私钥，如果有那么验证通过

早期执行方式的缺点

对于这样的交易模式，M N是固定的。不同的电商采用的M 和N可能不一样，这就给用户带来了很大的不便

使用P2SH的改良版

改良的地方在于将 输入脚本的复杂 转移进了 赎回脚本

即现在的用户只需要提供脚本和验证脚本的签名就行，至于N M的大小不用管

redeemScript:

M

PUSHDATA(pubkey 1)

PUSHDATA(pubkey 2)

...

PUSHDATA(pubkey N)

N

CHECKMULTISIG

input script:

X(无关元素)


```
PUSHDATA(Sig 1)
PUSHDATA(Sig 2)
...
PUSHDATA(Sig M)
PUSHDATA(serialized RedeemScript)
output script:
HASH160
PUSHDATA(RedeemScriptHash)
EQUAL
```

脚本执行

```
1
FALSE
PUSHDATA(Sig 1)
PUSHDATA(Sig 2)
PUSHDATA(seriRS)
HASH160
PUSHDATA(RSH)
EQUAL
2
PUSHDATA(pubkey 1)
PUSHDATA(pubkey 2)
PUSHDATA(pubkey 3)
3
CHECKMULTISIG
```

实例

交易 [bc26380619a36e0ecbb5bae4eebf78d8fdef24ba5ed5fd040e7bff37311e180d](#) 的第一个输入：

Input Scripts

```
ScriptSig: 0[] PUSHDATA(72)[3045022100f98068a026e2fc75cfeffe84bbac4223ed172df42bca01fd748a14bd960b1695022062c61a7f4f2a63a65d96b0feaf2a048bc2ca93e5
[304402201ce986e3fd780f4fe81f40ceb271a8ff34c3845e385b8424f8b20d1b91f1282102205dc71831baf5606f59d06b1d115bda3ec28817cdb4bf9df06643d550c30ef19301]
PUSHDATA1[5221027ca87e1aa2595ec7771afee8f8dc6efdb301b8370c4386731b4bd82247dc74a321022cc9874ba092095dda47a4e4edb1781c43c35b3ec0429ac005df37
```

push的最后一个数据是序列化的脚本，反序列化后得到：

```
2 027c...74a3 022c...c94b 0357...ce3a 3 CHECKMULTISIG
```

交易 [0ac29fc675909eb565a0984fe13a47dae16ca53fb477b9e03446c898b925ab6b](#) 的第二个输出：

Output Scripts

```
HASH160 PUSHDATA(20)[80cff499983050ec4268d749a1f898bec53e9fc2] EQUAL
```

➤ output script

```
RETURN  
⋮ [zero or more ops or text]
```

这种形式的output被称为：

Provably Unspendable/Prunable Outputs

➤脚本说明

假如有一个交易的input指向这个output，不论input里的input script如何设计，执行到**RETURN**命令之后都会直接返回false，不会执行**RETURN**后面的其他指令，所以这个output无法再被花出去，其对应的UTXO也就可以被剪枝了，无需保存。

一种特殊的脚本，使得这部分比特币无法使用（可以理解为销毁）。这个脚本就是用来证明销毁部分比特币的。

脚本应用

1. 销毁部分比特币，得到一些小币（AltCoin）
2. 向区块链里面写入一些内容（digital commitment）。比如向return后面的部分输入某个知识产权文本的哈希值。这样，等到这个知识产权产生争议的时候，就可以向公开文本，计算文本哈希值，对比证明自己在某个时刻就已经知道了这个知识产权。

为什么不用coinbase域？因为只有发布区块的节点才能够向coinbase里面写入内容。而任何一个用户都可以采用销毁很少一部分比特币的方法，向区块链里面写入一些东西。

这个脚本的好处在于，矿工看到这个交易的脚本之后，就直到这笔交易不涉及金额，就不把它放在UTXO里面，对于全节点比较友好。

实例

Transaction View information about a bitcoin transaction

090a9343da1158dce3634076f3df0ff868ea777c97b3b2cf6f3d85ca7ea1f7b2

No Inputs (Newly Generated Coins)

18cBEMRoxHq... (ViaBTC Bitcoin Mining Pool) - (Spent)
Unable to decode output address - (Unspent)

12.65566873 BTC
0 BTC

12.65566873 BTC

Summary

Size	257 (bytes)
Weight	920
Received Time	2018-07-05 12:34:39
Reward From Block	530572
Scripts	Hide scripts & coinbase
Visualize	View Tree Chart

CoinBase

(decoded) 038c1808162f5669e14254432fd4696e65642062792077676c312f2cfabe6d6d26a96a921daecbb032ae60989eca36e83b25265e31744b7137efaf6e191dc15304000000000000000010ba10e

Output Scripts

DUP HASH160 PUSHDATA(20)[536ffa992491508dca0354e52f32a3a7a679a53a] EQUALVERIFY CHECKSIG

RETURN PUSHDATA(36)[aa21a9ed29817b2b7b64c8e79ef0fa7214fa67135872eb28a5b5b1a6923f83a37c23bfc96]
(decoded) ♡♣♦♥ ♡[°L⓪y♣♠O♣q5♣. + ♡[[[]#♣:7♣. ♣♣



交易的同时，写入了一些内容

Transaction View information about a bitcoin transaction

1a2e22a717d626fc5db363582007c46924ae6b28319f07cb1b907776bd8293fc

1MQaYLeIR39TyN9PTxpAQcLBxFUqNHXx3M (0.05 BTC - Output)

Unable to decode output address - (Unspent)

0 BTC

0 BTC

Summary

Size	188 (bytes)
Weight	752
Received Time	2013-03-29 04:32:21
Included In Blocks	228596 (2013-03-29 14:18:58 + 587 minutes)
Confirmations	303536 Confirmations
Visualize	View Tree Chart

Inputs and Outputs

Total Input	0.05 BTC
Total Output	0 BTC
Fees	0.05 BTC
Fee per byte	26,595.745 sat/B
Fee per weight unit	6,648.936 sat/WU
Estimated BTC Transacted	0 BTC
Scripts	Hide scripts & coinbase

Input Scripts

ScriptSig: PUSHDATA(71)

[3044022055bcb36c829a614451787fe8c9bfb3798b683809b65b92037a015eccb5ff659702202461d2c708a4fd57c839e43634e8c02935d7b7d1db5b978432b0674c44645ec
PUSHDATA(33)[032c1ea520c25c4e66831cd395a3cd26f0e0a1472a3103fc8a4a63eff10e92d123c]



Output Scripts

RETURN PUSHDATA(20)[215477656e74792062797465206469676573742e]

(decoded) | Twenty byte digest

该交易没有任何货币的转移，只是写入了一些东西，转入的费用全部作为交易费转给矿工

脚本总结

1. 比特币的脚本语言很简单直接
2. 该脚本语言没有循环结构，虽然实现不了很多复杂的功能，但是也直接避免了死循环停机的可能
3. 比特币的脚本语言在密码学相关方面具有很强的优势

分叉

分叉非为两类：关于区块链状态意见不同造成的分叉（state fork），关于比特币版本协议意见不同造成的分叉（protocol fork）

StateFork

这种分叉的例子我们已经见了很多，比如同时挖到两个区块的分叉，以及恶意节点的分叉攻击（deliberate fork），不多赘述。

ProtocolFork

这种由于比特币版本协议意见分歧造成的分叉可以分为两种：硬分叉（hard fork）和软分叉（soft fork）

这两者简单总结来源：

新认老，硬分叉 更新协议的新节点认可（兼容）没有更新协议的老节点（挖出的区块），会造成硬分叉

老认新，软分叉 没有更新协议的老节点认可（兼容）更新协议的新节点（挖出的区块），会造成软分叉
不理解的话先向下看看这两者的来源和实例吧。

硬分叉（顽固派挡道）

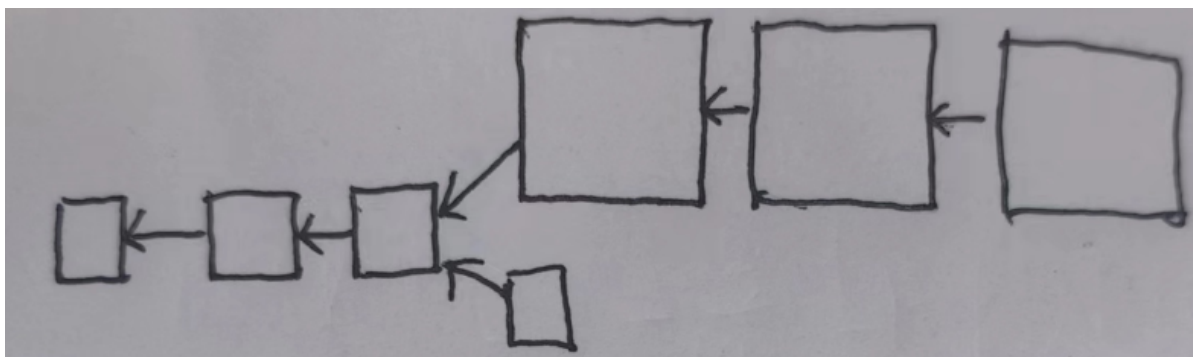
1. 如果有少部分节点不同意某个意见就会造成永久分叉
2. 这种分叉是永久存在的

实例

关于区块大小的调整

现在，我们假设大多数节点同意将区块大小改成不超过4M，少部分节点依旧坚持区块大小不得超过1M（按照算力大小）

首先，我们重申一下挖矿的共识规则，在最长合法链后面挖矿



1. 新节点认为，上下两个分叉都是**合法链**（注意，没说是**最长合法链**），因此在上下方都可以挖
2. 老节点认为只有下方的节点是**最长合法链**，因此只在下方挖

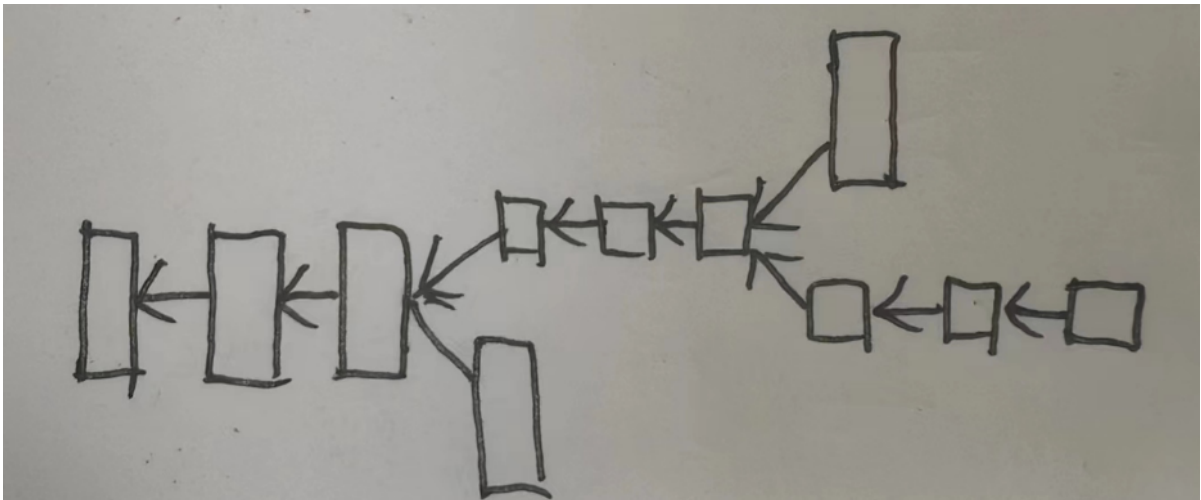
3. 这个时候就很有可能造成上下两条链旗鼓相当，如果还存在固执的老节点，这个分叉就永远不会消失。
4. 经典的硬分叉案例可以去看以太坊的ETH和ETC货币
5. 这时候有人会问了，这两条链上的货币要怎么算？至于是不是都认可，回滚还是其他方式，区块链会出具相关的协议来达成共识。

软分叉（欺负老同志）

还拿挖矿大小来举例

现在，我们假设大多数节点同意将区块大小改成不超过0.5M，少部分节点依旧坚持区块大小不得超过1M（按照算力大小）

我们再重申一下挖矿的共识规则，在最长合法链后面挖矿



1. 新节点认为，只有上方的链是合法的，而老节点认为上下方的链都是合法的
2. 现在，新节点只会在上方挖，上方这条链成为双方共识的合法链，而且它最长，因此就是**共识的最长合法链**！
3. 那么，老节点也会在上方的链上进行挖矿，而它挖出的大区块依旧会被分叉取代掉
4. 因此，老节点为了自己的出块奖励，势必要修改协议

实例

1. 关于在CoinBase区域（在Block_Header里面）内部写入UTXO表的根哈希值
这个是为了方便轻节点验证全节点提供给自己的资产的信息是否真实（类似于Merkle proof）
也就是老节点允许你向CoinBase区域（反正原来就是随便写的），而新节点不承认不在CoinBase里面写根哈希值的区块
2. P2SH脚本的推行
老节点只进行P2SH的第一阶段验证，新节点进行两个阶段的验证
因此，对于第一阶段过了但是第二阶段没过的脚本，新节点不认，而老节点认可。

分叉总结

现在能不能理解上面的总结？

新认老，硬分叉 更新协议的新节点认可（兼容）没有更新协议的老节点（挖出的区块），会造成硬分叉

老认新，软分叉 没有更新协议的老节点认可（兼容）更新协议的新节点（挖出的区块），会造成软分叉

问答回顾

提出下面几个问题，回忆下之前涉及的知识（欢迎在评论区交流补充）

转账地址类

A给B转账，B不在线，转账可以进行吗？

可以！

A转账给B，但是B节点从来没有被其他节点听说过，可以转账吗？

可以！

写错收款人地址怎么办？

没办法，除非你联系对方，他良心发现给你转回来

转账的地址不存在怎么办？

恭喜你，成功销毁了部分比特币！只有你和全节点受伤的世界达成了！

什么？问什么全节点受伤？因为那个不存在的地址将会永久存在UTXO模型中

记不记得销毁比特币的交易？这个交易最后return的是错误的值，那么它问什么能被判断为合法交易并且上链呢？

因为验证的是比特币来源的输出是否合法，和本交易输出没有关系

简单理解就是return管理的是这笔钱怎么用，是在用它的时候要验证的

为什么？

1. 区块链的账户创建是在本地进行的
2. 区块记录的只是转账的交易，只验证你有没有钱转出去，不验证接收者

找回密码类

私钥丢了怎么办？

1. 如果是正常创建的账号，那么你又达成了和全节点一起受伤的世界（dog）
2. 如果是在数字货币交易所创建的账号，私钥（密码）是可以找回的，因为数字货币交易所相当于一个中心化金融机构

但是数字货币交易所也不是绝对安全的，毕竟你很可能碰上Mt.Gox

私钥被泄露怎么办？

如果你发现自己账户存在一些不是你操作的交易记录，很可能是你的私钥泄露了

这时候尽快将你的资产转给另一个账户

为什么区块链被某些不法分子利用来转账？

因为资产不可冻结

总结

区块链是去中心化的

小丑小偷类

我们能不能偷走一个人的区块，抢先发布获得收益？

不能！因为**铸币交易的地址是挖出区块的人的**

上述问题的答案也能用来解释为什么矿工不能私藏区块不让矿主知道，并且自行发布获得收益

我该向谁支付交易费？

交易费的input>output，两者的差额作为交易费。因此不需要转交易费给谁，只需要多给一部分，谁给你上链，谁就获得这部分钱。